

Interrim



Project Title Interim Report

TU856
BSc in Computer Science

Cianán Finn
C22393223

Supervisor
Fadoua Rafii

School of Computer Science
Technological University, Dublin

Date

Abstract

This interim report outlines the ongoing development of my Final Year Project, “Mapping the Spoken Word: A Geo-Spatial Platform for Voice Recognition and Analysis”. The project aims to build a machine learning model capable of accepting Hiberno-English speech samples, classifying them regionally, and mapping the predicted origin onto a geographic map of Ireland. My project brings together three distinct fields: machine learning for accent recognition, web-mapping technologies, and regional linguistic analysis.

My report presents the background motivation for the project, the related research I have conducted, an analysis of the system requirements, the architectural and design decisions I have made, and my initial implementation and testing plans.

By the end of this interim stage, a functional prototype of the system will be established, demonstrating all major components of the application. The system is being developed with a modular architecture to ensure low coupling between components, supporting iterative experimentation and enabling steady, maintainable progress toward the final system.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

A handwritten signature in black ink, appearing to be 'Cianán Finn', written over a horizontal line.

Cianán Finn

23/11/2025

Acknowledgements

I would like to acknowledge the life of Simon Adnan, a classmate of mine who sadly passed away during my writing of this report. He was truly a lovely man, and each interaction with him left me with a smile on my face. I truly hope the best for his family. His life left a lasting impact on me, may he Rest in Peace.

Contents

1. Introduction.....	1
1.1 Project Background.....	1
1.2 Project Description.....	1
1.3 Project Aims and Objectives	1
1.4 Project Scope.....	2
1.5 Thesis Roadmap	2
Chapter 2: Literature Review	2
Chapter 3: System Analysis.....	2
Chapter 4: System Design	2
Chapter 5: Testing and Evaluation	2
Chapter 6: System Prototype	3
Chapter 7: Issues and Future Work	3
2. Literature Review.....	4
2.1 Introduction	4
2.2 Alternative Existing Solutions	4
2.3 Technologies Researched	4
2.4 Other Research	5
Hiberno-English Linguistics and History	5
Linguistic Mapping.....	5
2.5 Existing Final Year Projects.....	5
2.6 Conclusions	6
3. System Analysis.....	7
3.1 System Overview	7
3.2 Requirements Gathering.....	7
Key Stakeholders	7
Requirements Collection	7
Initial Requirements:	8
3.3 Requirements Analysis.....	8
Frontend Layer	8
Backend Layer.....	8
Machine Module.....	8
Database Layer	9
Storage Layer.....	9
3.4 Risks in System Analysis	9
3.5 Initial System Specification	9

Functional Requirements:.....	9
Non-Functional Requirements:.....	10
Architectural Design:.....	10
• Frontend Layer:.....	10
• Backend Layer:	10
• Machine Learning Module:.....	10
• Database Layer.....	10
• Storage Layer	11
3.6 Conclusions	11
4. System Design	12
4.1 Introduction	12
4.2 Software Methodology	12
4.3 Overview of System	12
Logical Architecture	12
• Frontend Layer.....	12
System Deployment.....	14
• Backend Container	14
• Database Container	14
4.4 Design System.....	14
Frontend Design	14
Backend and API Design.....	15
Machine Learning Module	15
Database Design	16
• Region	16
• Prediction	16
Storage Design.....	16
Data Flow Design	16
4.5 Conclusions	17
5. Testing and Evaluation	18
5.1 Introduction	18
5.2 Plan for Testing	18
The Frontend.....	18
The Backend.....	18
The Machine Learning Module	18
The Database	19
The Storage Layer	19

5.3 Plan for Evaluation.....	19
The Machine Learning Module	19
• Accuracy and Performance	19
• Prediction Consistency	20
• MFCC Feature Validation.....	20
• Known-Origin Audio Testing	20
• Classifier Reliability	20
The Frontend.....	20
• Map Accuracy	20
• Markers	20
• Usability.....	20
The Backend.....	20
• Error Handling	20
• Database Consistency	20
• Storage Integrity.....	20
The Whole System.....	21
5.4 Conclusions	21
6. System Prototype	22
6.1 Introduction	22
6.2 Prototype Development.....	22
Frontend.....	22
Backend	24
Machine Learning Module	25
Database Prototype	26
Storage Layer.....	27
6.3 Results	27
Frontend Tests	27
Backend Tests.....	29
Machine Learning Module Tests	31
Database/Storage Layer Tests	34
6.4 Evaluation.....	35
Frontend.....	35
Backend and Database.....	36
Machine Learning Module	36
6.5 Conclusions	36
7. Issues and Future Work	37

7.1 Introduction	37
7.2 Issues and Risks	37
Dataset Limitations.....	37
• Regional Imbalance	37
• Demographic Skew	37
• Accent similarity	37
Overfitting Risk	37
• Microphone Quality	37
• Background noise in the Dáil.....	37
• Phrasing used in Political Speeches	37
Frontend Features	37
• MediaRecorder.....	37
The MediaRecorder has yet to be implemented, with users only being able to upload data as files.....	37
• Results List.....	37
• Clunky User Interface	37
Time.....	38
7.3 Plans and Future Work.....	38
The Frontend.....	38
Potential Future Frontend UI.....	38
The Backend.....	39
The Machine Learning Module	39
The Database	40
7.3.1 Project Plan with GANTT Chart	40
References.....	1
• Appendix A: System Model and Analysis.....	2
• Appendix B: Design.....	1
• Appendix C: Prompts Used with ChatGPT	1
• Appendix D: Additional Code Samples.....	1
• Appendix E:	1

1. Introduction

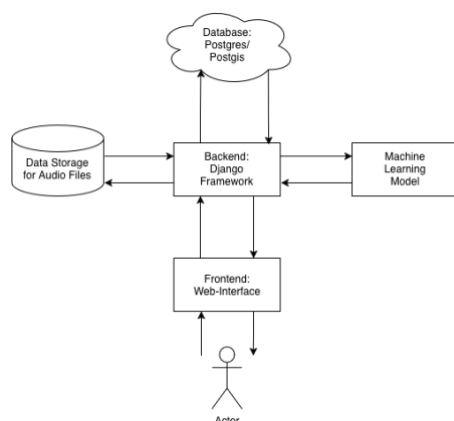
1.1 Project Background

The Hiberno-English accent is the official name of what we normally refer to as the Irish accent. The name comes from our past, back when the Romans called our island Hibernia, meaning the land of winter; which comes from the same root as the word “Hibernate”. English is usually classified into either British-English or American-English, which are broadly speaking identical, only differing when it comes to small spelling alterations, such as American-English dropping the “U” from words such as “Flavour” becoming “Flavor”, and some words having different names, such as “Boot”, meaning the back of the car, being “Trunk” in the US.

Hiberno-English is quite different in a truly fascinating way. The dialect originates from the Irish people directly translating from their mother tongue, Gaeilge, the Irish language. Gaeilge penetrates our day-to-day vocabulary even today in Ireland, even in Dublin (where According to von Münchhausen’s diary, English was the common tongue all the way back in 1591 (von Münchhausen, 1591). Common phrases we have all heard, such as the ugly word “Mot” meaning girlfriend actually has a nicer root from a traditional way to refer to one’s girlfriend “Mo Chailín Maith”, translating to “My Good Girl”, or the remnant of Gaeilge not having a word for yes or no, so interactions like “*Are you coming home soon?*” – “*I am.*” remain common.

My own dialect has always fascinated me, and having the chance to focus my Final Year Project on a personal passion, and intertwine it with my academic studies has been extremely rewarding. The Hiberno-English’s uniqueness makes accent recognition research a challenging but exciting area of study. This rarity of existing research into the dialect, and of specific datasets of the Hiberno-English accent provides a strong incentive to create my project proposal, creating a distinct research contribution.

1.2 Project Description



My Final Year Project is called “Mapping the Spoken Word: A Geo-Spatial Platform for Voice Recognition and Analysis”. My goal is to build a machine learning model, that is trained on regionally specified Hiberno-English accents that I have created the sample set of data for. This data will be stored on a database for querying. A user will then be able to input speech data into my web-app, which will then predict where the speech data originated from, creating coordinate results which will then be displayed on a Leaflet map of Ireland.

1.3 Project Aims and Objectives

1. Research machine learning techniques and find those that apply to this project.

2. Research web-mapping techniques and find examples that apply to this project.
3. Find/Generate a sample set of high-quality Hiberno-English.
4. Divide the data into training data, testing data and ____ data.
5. Break the data up into MFCC features to get numerical values for the data.
6. Create metadata for the sample set of data.
7. Use the metadata with the data to train a machine learning model.
8. Test the model to ensure the best results possible.
9. Create the application: Front end to accept user input and output data, back-end to handle requests, database to store the data, machine learning model to query.

1.4 Project Scope

The aim of this project is to research, design and implement a machine learning system that can regionally classify Hiberno-English accents with high accuracy and consistency. While my full system includes a Frontend for receiving input from the user, as well as visualising the Machine Learning Module's results on a map, my main focus is on the Backend components: Audio Processing, Feature Extraction, Machine Learning Classification and Data Management.

My scope involves the development of a Backend module, that can be passed an audio sample, pass it to the machine learning classification, which extracts the MFCC features, which will be used for accent classification, which will finally be used to predict the geographic coordinates to be displayed on a map of Ireland.

My scope does not delve deep into linguistic science, creating a production-grade UI, or achieving a 100% classification system. I don't plan to expend much project time into cloud-based deployment, with any cloud elements being secondary to my existing scope.

My focus is on building a functional, well-designed system that shows the feasibility of regionally mapped accent recognition with a limited dataset.

1.5 Thesis Roadmap

Chapter 2: Literature Review

Chapter 2 describes my background research prior to beginning this project, showing existing research and technology, as well as any related systems I could find.

Chapter 3: System Analysis

Chapter 3 shows the system analysis of my project, where I identify the stakeholders and the requirements, which leads me to my initial system specification.

Chapter 4: System Design

In this chapter is where I design my system. I explain my chosen methodology, architecture and each module's design decisions.

Chapter 5: Testing and Evaluation

Here is where I outline my testing and evaluation procedures, to ensure all errors and issues are accounted for in each part of the system.

Chapter 6: System Prototype

Here is where I present my prototype that I have created based on all previous chapters. Its purpose is to demonstrate the early implementation of my application and any results I can currently produce.

Chapter 7: Issues and Future Work

In chapter 7 I look to the future and indicate what issues I have yet to fix and my planned future work I wish to achieve to complete the full finished system.

2. Literature Review

2.1 Introduction

In this chapter I plan to review any existing research, tools and any other work that relates to a geo-spatial accent recognition systems. My project combines three distinct fields: web-mapping, accent recognition machine learning, and regional dialect analysis. I plan to show in this section how there is a gap in software that combines all three together.

2.2 Alternative Existing Solutions

Vocal Image

Vocal Image is an accent training software designed to assist people improve their pronunciation and communication skills. Their accent test application is the closest system to my idea, as it accepts speech data and returns an estimated country of origin displayed on a world map. While it is a good resource for front-end design inspiration, it differs from my project as it doesn't distinguish accents regionally. I tested other accent checkers, but I found that most produced very inaccurate results (Vocal Image ,2023).

Google Speech-to-Text

Google's Speech-to-Text API is one of the many professional high-accuracy speech recognition APIs. Other options include Amazon's 'Transcribe' or Open AI's 'Whisper'. These systems are very good for transcription and language extraction, some even able to discern between British-English and American-English, but they don't show any regional classification or any visualisation of their speech recognition data. Since they are closed-source, there is no way to access the data either (Google Cloud, 2023).

Mapping Tools

ArcGIS is a web-application that lets users create and analyse map visualisations. It is a fantastic resource for any GIS based projects, but it requires external data to produce any visualisations, where my project differs (Esri ,2023).

2.3 Technologies Researched

Speech Processing and the Machine Learning Module:

- **Python** will be the main language of my project as it has a fantastic selection of audio processing libraries.
- **Librosa** (McFee et al., 2015), **torchaudio** and **scikit** – These libraries will be used thanks to their audio learning and audio feature extracting features.
- **MFCC** (Mel-Frequency Cepstral Coefficients) will be how I convert speech data to numerical values to be used by the machine learning module (Journal of Engineering, IOSR 2014).
- **Wav2Vec2** is a deep-learning model that is especially suited for analysis of small dataset models, which suits this project due to the limited availability (Baevski et al., 2020).

Backend:

- **Django** will be used to provide a stable and easily modifiable backend (Django Software Foundation, 2023).
- **PostgreSQL and PostGIS** to store the metadata and geographic data (PostGIS Project, 2023).
- **Docker** to pack the whole system into one container so I can use it on different platforms (Merkel, 2014).

Frontend:

- **Leaflet.js** is the mapping library I will use as it is the one I have the most experience with, and it makes it easier to add prediction markers on an interactive map (Leaflet, 2023).
- **HTML/CSS/JS** for a simple front end to display the information.
- **OpenStreetMap** will be the source of the map tiles I will use for this project (OpenStreetMap Contributors, 2023).

2.4 Other Research

Hiberno-English Linguistics and History

I read Raymond Hickey's (Of the University of Limerick) 'Irish English Studies: An historical overview' (Raymond Hickey, No date). This was a fantastic introduction to the history of the Irish dialect, giving me insight into the fascinating history it has. UCC had a diary from 1591, authored by the German Explorer 'Ludolf von Münchhausen' (von Münchhausen, 1591) in its online archive, which gave me more knowledge about the history, supporting the information that Hickey had in his presentation.

Linguistic Mapping

[Canúint](#) (RTÉ, 2025) is a RTÉ/DCU project to document Gaeilge speakers around the country, having markers on a map of Ireland, each producing an audio clip of a sample of a native Gaeilge speaker from their archive. Although it focuses on Gaeilge rather than Hiberno-English, it demonstrates an effective way to present regional speech data through a simple, interactive map. Its design offers useful inspiration for the visual layout and user interaction patterns in my own project.

[Locallingual](#) is a project by David Ding (Ding, D. 2016), who created the interactive website that allows for users to click a country. The country is then zoomed in upon, and divided into sub-regions, allowing the user to select their sub-region of origin, and record a sample of their accent. The app doesn't perform any linguistic analysis, but it demonstrates a clean way to organise audio geographically. Its map-based interface provides useful inspiration for my frontend design, showing how regional audio data can be visualised simply and effectively.

2.5 Existing Final Year Projects

"A Comparative Study of Machine Learning Classifiers for Speaker's Accent Recognition" by Hasibul Hasan Sabuj, Paul Richie Gomes, Kazi Ehsanul Mubin, Akram Hossain, Samin Yeasar Seaum, Md Tanzim Reza and Md. Golam Rabiul Alam.

This research paper explored accent classification of the English language. They trained their deep-learning machine learning model on UK, US, Spanish-English, French-English,

German-English, Italian-English data, allowing new data to be classified into one of these six accent classifications. Throughout their research they tested 8 machine learning classifiers, achieving their highest accuracy, 95.28%, with a Random Forest approach, but 5/8 had an accuracy of 90% or more; only the Gradient Boost approach failed (H. H. Sabuj et al., 2023).

2.6 Conclusions

Through my research in this chapter, I have found that while many other systems exist that have elements that overlap with my project, none exist that combine all three of accent classification, regional-accent analysis and a mapped visualisation of the result. Vocal Image demonstrated a system with accent classification and mapped a visualisation of the results but had no regional-accent analysis. Speech Recognition APIs like Google's Speech-to-Text had fantastic accent classification accuracy, but due to its closed source nature, there was no visualisation of its speech data analysis.

In my research, I found many map-visualisation tools such as ArcGIS online, which are fantastic for representing geographic data, but require all data to be inputted into the system.

Other academic studies were quite closely linked to what I am doing. I found "A Comparative Study of Machine Learning Classifiers for Speaker's Accent Recognition" a insightful text, where they clearly demonstrate their problem approach, and their accurate results. Using these previous research papers, I will be able to follow in their footsteps and continue to contribute academically through my own research.

Other research inspired me to pursue this project. My research into Hiberno-English gave me a refreshing perspective on my project, one that had been lost to cold, quantitative data analysis. Researching other forms of Linguistic-Mapping gave me inspiration of how to create my frontend as they beautifully displayed geographic data.

In summary, I have found that there is a gap currently for a system that combines accent-classification, regional-accent analysis and map-visualisation; and with the research that I have completed, there are many systems that I can rely on to inspire me to create this unified system.

3. System Analysis

3.1 System Overview

For my system, the user perspective will be clear and clean, allowing for a smooth experience while using it. Upon initially loading the application, the user will be greeted by a webpage divided into three clear sections: the user input section, a section which lists data by previous users split by region, and a map section with markers representing this previous user data.

Upon clicking the input button, the user will be prompted to speak into the computer microphone. The input section will update to show the recording has started. Once the user is finished speaking, they will click a button to stop recording, and this sample will be sent to the backend for analysis. Once a location estimate has been determined, the screen will update again. The users estimate will be shown in the list and as a new icon on the map. The user will be displayed a selection of buttons: a buttons to listen to the recording and another to retake the test.

3.2 Requirements Gathering

Identify and consider key stakeholders
Describe how to collect requirements
Collect initial requirements

Key Stakeholders

The key stakeholders I have identified in this project are:

- The Irish General Public – The end users will be the Irish general public as the app is designed for those who wish to record themselves speaking and seeing their estimated regional accent on a map.
- Researchers/Linguists – Anyone with an interest in visualising linguistic data based on geographic variation has a stake in this project.
- Project Tutor – My project tutor has a stake as they recommend alterations and provide feedback.
- Me – I am responsible for the entire project.

Requirements Collection

Due to the nicheness and high-technicality of my project, I decided against conducting a research survey. In place of this I gathered requirements by a selection of other methods:

- Review of Existing Tools
I reviewed existing tools (Vocal Image, Canúint, Locallingual) to understand the desirable user-interface and what features are expected of web-applications within the accent-classification/linguistic-visualisation web-app landscape. This helped me identify the expectations of the user-experience and map-based visualisation.
- Review of Academic Previous Research
I studied previous academic papers to get an understanding of what technical requirements are typically necessary for machine-learning accent-classification systems. It was here that I expanded my knowledge of machine learning approaches. I learned which approaches worked best with small datasets, with MFCC features and with noisy dataset.

- Research on Datasets

Finding a regionally divided Hiberno-English dataset was one of the most difficult sections in regards to requirements gathering. There was a significant amount of Hiberno-English speaking data online, but there was almost nothing regionally divided, making the data useless for my project. After a lot of searching I found a GitHub directory with a small (~190) of ~15 second clips of the Irish Dáil in session with a tidy csv of metadata. This was fantastic for my original prototype, and led me to creating my own dataset and metadata of ~1500 30 second clips, which I plan to expand in the future.

- Researching Web-Mapping Practices

I gathered requirements of the web-mapping aspect of my project by revising the notes and labs from my “Advanced Web-Mapping” class. This allowed me to understand what software packages and libraries will be essential in the design and implementation of my application.

Initial Requirements:

1. The system needs to accept user-recorded or uploaded speech.
2. The audio will be processed into MFCC features for analysis.
3. A machine learning model will classify the regional origin of the samples speaker.
4. The predicted results must be stored, along with metadata.
5. A map must display the previous results and update dynamically.
6. The frontend needs to remain simple and intuitive.
7. The backend must handle the API requests, audio handling, and model inference.
8. The database supports both relational and spatial data.
9. The system will operate modularly so each component can be tested and improved independently.

3.3 Requirements Analysis

Develop an initial systems model through analysis of requirements

After reviewing these requirements, I have divided my project into four:

Frontend Layer

This layer will be simple and user friendly, showing an interactive map, a list of clickable previous sample’s metadata and a section to accept user input (speech data). I will do this using HTML/CSS/JS and Leaflet.js for the map.

Backend Layer

The backend of my application must act as a router for passing the uploaded audio to the data storage, to the machine learning model, which passes the metadata to the database, which is then passed to the frontend to be displayed to the user. The backend must facilitate this passing of data around the other modules. It must be set up to allow for RESTful APIs, so that there are clear endpoints to reference the data. For this I plan to use Django.

Machine Module

This layer will need to be able to accept speech data. It will then extract the MFCC features (Journal of Engineering IOSR, 2014) from it, allowing it to be converted into numerical values to be analysed by a selected model (e.g. Random Forest or KNN prototype). The metadata and estimated coordinates will then be output and stored in the database.

Database Layer

Any extracted metadata and estimated coordinates will be stored in the database for easy reference by the frontend. It must be able to also store regional geography to allow for filtering by region, as well as prediction history so that previous results can be displayed on the map as well. The software I have decided on is PostgreSQL with a PostGIS extension for the geometric data.

Storage Layer

The storage layer is simply to store the audio samples that will be used for training/testing the machine learning model, as well as audio samples that have been collected by users of the application.

3.4 Risks in System Analysis

The section with the most risks in my project is the data and the machine learning model. These risks are:

- **Dataset Imbalance** is one risk I am particularly concerned with. As of 2024, there are 174 seats in the Dáil Ireland, but only 44 or 25% are women. This is an increase from the previous Dáil where only 23% were women. This dataset skewing towards men may imbalance my data due to some areas only having male speakers within my data. Another issue is that speakers in the Dáil tend to have a more refined form of English comparing to the constituents they represent, causing the data to be imbalanced toward more formal speaking.
- **Overfitting Risk** is an especially risky element of my project. My project has a small, self-collected dataset of Hiberno-English. Due to this, and the more subtle differences between different Irish accents, there is a danger of the machine learning model may learn non-human characteristics (such as mic-quality, Dáil background noise, etc). The model may accidentally treat these as features for accent-classification. This risks causing the model to perform well on training data, but worse on new data.
- **Irish Accents May Overlap Phonetically.** Most of accent-classification research deals with classifying accents on large geographic areas, such as by country, or by east/west coast. This allows the accents to have very distinctive characteristics which makes for more noticeable features. My project deals with electoral districts, causing the geographic distance from one to another to be very close, making the two very similar.

3.5 Initial System Specification

Functional Requirements:

1. System must load database and frontend.
 - a. Database is queried and previous estimates are displayed on the map and within the list.
2. System must accept user speech data.
 - a. Allow users to record speech they wish to be analysed within the browser,
 - b. Or, Allow users to upload speech data they wish to be analysed.
3. Extraction of MFCC features.
 - a. Data to be converted from raw audio into numerical features for analysis by the machine-learning model.

4. Pass extracted data through machine-learning model to perform accent-classification.
5. Take result and store sample data and metadata in database.
6. Take result and update frontend display.
 - a. Show predicted coordinates.
 - b. Update list with sample estimate.
 - c. Update map with new marker in location of sample estimate.
7. Sample is stored within the database, showing when the application is used to conduct another accent-classification.

Non-Functional Requirements:

1. The system must process the predictions quickly.
2. The interface should be simple and user-friendly.
3. The system should be packaged into one container using docker to allow it to work across platforms.
4. The database needs to allow for spatial queries, PostGIS.
5. The system needs to store the user data securely.

Architectural Design:

I have decided to approach my development of the app with a modular approach, separating each layer to cause the lowest possible coupling. It makes the code easier to debug as each part can be tested individually, and it allows me to maintain and change the application without disturbing the entire system(e.g. I could change the machine learning model from Random Forest to KNN without the frontend being affected)

- **Frontend Layer:**
The frontend of my application is for the user to interact with. It will allow for users to upload/record speech data and output a map of previous samples as well as the current user's results. I will use Leaflet.js for the mapping aspects of the frontend, and HTML/CSS/JavaScript for the rest.
- **Backend Layer:**
My backend is to handle the RESTful API endpoints, routing data around the application's different modules, between the frontend, machine learning module, the database and the data storage. I have decided that using Django is the best for this, as it has a very developed community and I have previous experience with it.
- **Machine Learning Module:**
This module will perform the MFCC feature extraction and data classification using the model I find performs best (currently Random Forest).
- **Database Layer**
This layer will hold the metadata, the geographic boundaries and all of the previous prediction samples. For this I will use PostgreSQL with a PostGIS extension.

- **Storage Layer**
This layer will simply act a storage for the raw audio data of the existing dataset as well as the new data collected. For my current prototype, a local storage on docker will suffice.

3.6 Conclusions

In this section I used my previous research to guide me in my planning of the requirements of this application. Using a mix of academic literature, commercial applications, discussing issues with my tutor and general research, I identified all requirements to create a successful application. Due to having a firm grasp on these requirements, I was able to successfully create a fully thought through application architecture plan.

4. System Design

4.1 Introduction

This chapter will explain how I used the system requirements I gathered in chapter 3 to create a in depth System Design for the implementation of the application prototype. I will outline the iterative, short-cycle approach I will use to develop my application, as well as the 5-layer modular approach I have decided to advance with. The chapter will go into depth of each layers' logical and physical architecture as well as the design decisions that I have made for each part of the system. My goal is to ensure the system is well structured, scalable as my project grows and easily maintainable.

4.2 Software Methodology

Although I initially planned on using the Waterfall linear methodology, I found that it was too rigid for my project due to the continuous experimentation of my machine learning module and my evolving dataset. I found that an iterative approach works better, allowing me to continuously create prototypes, re-adjusting any section as needed as my application evolves.

My early developments began with experimenting with feature extraction and testing different machine learning models on basic datasets. As I got results and learned, I was able to adapt my approach to better suit my application's shifting requirements.

Each development cycle consists of four short repeatable stages:

1. Planning – First I decide on a small set of goals I wish to accomplish in this cycle (e.g. add a map to the frontend).
2. Implementation – Make needed changes to accomplish these goals.
3. Evaluation – Review results of the changes and conduct tests to identify any issues.
4. Revision – Refine any changes and fix any identified issues for next cycle.

GitHub has been my chosen software for version control. As I iterate through each cycle, GitHub records each change and allows me to log what I have done. Whenever I encounter any major issues, I have been able to revert to a previous version and revise my stage plan to address the known issue.

This methodology has been perfect for me, being flexible enough to allow for experimentation with my machine learning model, with altering the frontend to improve user experience, and to allow my dataset to evolve, while I have steadily maintained steady progress towards the final system.

4.3 Overview of System

Logical Architecture

I have designed the system with a 5-layer modular architecture. Each layer has been ensured to have the lowest coupling possible, leaving each module to be worked on and tested independently. The five layers are:

- Frontend Layer
The frontend layer has been created using HTML, CSS and JavaScript, with Leaflet.js (Leaflet, 2023) being used for the interactive map element.

Its requirements:

- Use the browser's "MediaRecorder" API to allow for the recording and/or allow for uploading of speech data by the user.
- There must be buttons to start recording audio, to stop recording audio, to listen to audio recording and to re-record audio
- The audio recording must be saved and sent to the backend for processing by the frontend.
- Previous prediction results must be displayed on the map and the list.
- The predicted results must be returned to frontend to be displayed to the user, updating the list and the interactive map with the new data.

- Backend Layer

The Django (Django Software Foundation, 2023) backend of the application is to act as the coordinator of the system, allowing for communication between the different layers.

It provides:

- RESTful API endpoints for the audio upload, handling the prediction requests, and retrieving the stored results.
- It routes data to the machine learning module, and then to the database.
- It converts the machine learning module metadata output into JSON for storage.

- Machine Learning Module

The machine learning module is a component that is only interacted with by the backend. It will be implemented in Python due to the extensive amount of libraries available for data analysis.

It is responsible for:

- Extracting MFCC features from the raw audio.
- Running the results through the classifier I will select (Currently Random-Forest).
- Creating a prediction of the samples coordinate values.
- Returning this result to the backend.

- Database Layer

My system uses PostgreSQL for my structured data and PostGIS (PostGIS Project, 2023) for any spatial data.

It stores:

- Polygons of regional boundaries.
- Metadata of the predictions (e.g. Predicted coordinates, file-path to stored audio file).
- Metadata of the audio (e.g. Speaker, County of Origin, Current County)

- Data Storage Layer

This layer contains all the raw audio collected from datasets and from user samples.

- During the prototype phase it will store the data in the local filesystem directory.
- In the future I may implement a cloud storage approach.

Consider and briefly the system deployment or production environment

System Deployment

I recently learned how to deploy application using docker, and I found it to be the best method for the deployment of my system. I will split the application into a multi-container setup:

- **Backend Container**
This container will run Django, with all REST APIs being provided here. The machine learning module will be integrated into this container.
- **Database Container**
The Database container will run PostgreSQL with a PostGIS extension. It will store the metadata, the regional polygons and the prediction results.

4.4 Design System

Use design methodology to create design

In section 4.2 I decided upon using an iterative, short cycle methodology to design my system. Each cycle targets specific fixes in the system, slowly building upon each iteration to produce the final result.

Frontend Design

The frontend is being designed to be a lightweight, browser-based application that is simple and clear for the users. I am implementing it using HTML/CSS/JavaScript, as well as Leaflet.js for the interactive web-map implementation.

User Interface Structure:

- **Input Section**
This section of the UI will be used to receive input from the user. It will display the controls for recording audio. It will also accept speech file upload. This section will provide visual feedback on the current stage of data processing, i.e. “Recording...”, “Extracting Features...” etc.
- **List of Results**
This section will display a list of previous sample results divided by region. It will display a coordinate result, which may be clicked on to show a popup of the metadata of that sample. As the user uploads a sample, the prediction result will appear here.
- **Map Panel**
This section will contain an interactive map of Ireland, with markers representing previous prediction results in their corresponding geographic location on the map.

The map tiles will be sourced from OpenStreetMap and Leaflet.js will allow for the necessary interaction from the user. Once a sample is tested and predicted, the user will receive an updated Frontend, with the result of their sample displaying on the interactive map as a highlighted icon in the geographic location predicted. This icon will be clickable to view the objects metadata.

Frontend Communication:

The frontend will be configured to work with RESTful APIs in mind, so that it communicates with the backend using `fetch()` calls, which will reference the APIs I will speak about in the Backend and API Design section.

Backend and API Design

The backend of my application is being built using Django. I have chosen Django as I have previous experience using it, so there is no learning curve needed, reducing production time and error risk, and its stability, its modular architecture and thanks to it being written in Python, it easily integrates with machine-learning libraries. The backend is the central communication module, allowing for the other modules to communicate seamlessly.

The backend's API endpoints are:

- POST `/api/predict/` – Submit audio sample for classification.
- GET `/api/predictions` – Retrieve previous classifications in a JSON format.
- GET `/api/regions/` – Fetch the polygons representing the regions as GeoJSON.

And the backend's request handling flow is:

1. Backend receives audio from the user.
2. The sample is stored in the storage layer.
3. The sample is then passed to the machine learning module for prediction.
4. The machine learning module extracts the samples features and creates a prediction, passing this metadata to the backend for storage in the database.
5. The backend retrieves this metadata from the database and displays it to the frontend.

Machine Learning Module

I am implementing the machine learning module as independent module within my Django project so that it can evolve and adapt as I experiment and the dataset changes without affecting the rest of the project. Its role is to extract the MFCC features from the raw audio and create an estimate of where the samples speaker originates from.

Feature Extraction:

- For the prototype, I used Librosa (McFee et al., 2015) as it is a rich audio processing audio python library. It allows me to convert the speech data into numerical data by extracting Mel-Frequency Cepstral Coefficients (MFCCs).
- Using `yt-dlp`, I downloaded the data trimmed and scaled.

Machine Learning Models:

For the prototype I am using Random Forest (Sabuj et al., 2023) as my chosen classifier. I have trained it on the extracted MFCC features from the Dáil mini-dataset.

I have designed this module with modularity, simplicity and expandability in mind, allowing for the module to change and adapt as needed by my app's development, while keeping the input/output of the module simple to avoid issues.

Database Design

I am implementing my database using PostgreSQL with a PostGIS. This setup is perfect as my design as it handles the relational and spatial data perfectly. I decided to use PostGIS as it allows for spatial querying, essential for the necessary regional dividing I will use for my visualisations. My decision to separate the prediction result from the raw audio allows for much quicker query processing times as the tables are much smaller and lighter.

The database will have two tables:

- **Region**

With columns **id**, **name** and **region_polygon**(PostGIS).

This table is to hold any information related to the region I will choose, either Constituency, County or Province.

- **Prediction**

With columns **id**, **audio_file_path**, **timestamp**, **predicted_coordinates** and **confidence_score**.

This table will be used to store the information generated by analysis of the received sample and the prediction result returned by the machine learning module.

Storage Design

The storage layer of my application stores and maintains all the raw audio files used for the machine learning module predictions and training.

For the prototype I am trying to keep this section simple to minimise risk of any issues. Therefore, I am storing all data locally within a docker-mounted directory. Each file will have a corresponding reference stored in the database.

In the future, I may upgrade the file storage system to a cloud-based system. The options I have briefly researched are:

- AWS.
- A self-hosted dedicated media server.
- DigitalOcean Spaces

Data Flow Design

The data flow design I have settled with is the following:

1. The user uploads or records audio.
2. Frontend sends this data to the Backend.

3. The Backend stores the audio into the System Storage.
4. The file path is passed to the Machine Learning Module.
5. Machine Learning Module extracts the MFCC features and predicts the coordinates of origin of the speaker.
6. The Machine Learning Module passes this data as JSON to the Backend, which stores it in the Database.
7. The Frontend updates with the new data from the Database.

4.5 Conclusions

In this chapter, I used the requirements I gathered in chapter 3 to create a thorough design for how my system will work in practice. By choosing an iterative, short-cycle methodology, I've kept the development flexible enough to deal with the experimentation in the machine learning module and the evolving nature of my dataset. This approach lets me adjust any part of the system without affecting the whole project.

The five-layer modular architecture I settled on gives each part of the system a clear purpose: the frontend handles the user interface and mapping, the backend routes data and exposes the APIs, the machine learning module performs the actual predictions, the database manages all structured and spatial data, and the storage layer stores the raw audio. Because these layers are loosely coupled, each can be worked on, tested, or replaced independently, which will make the rest of the project far easier to manage.

Overall, the design decisions made here give the system a strong foundation before I begin implementing the prototype. With the architecture, data flow, and development approach all pre-planned, I can now build and test each component with confidence.

5. Testing and Evaluation

5.1 Introduction

In this chapter I will outline how I will test and evaluate each part of the system I have so far designed. Due to my usage of a modular architecture, my application can be divided into individual modules fit for testing separately. This will allow me to quickly identify, repair and reintegrate modules as issues arise during development.

In the subsections of this chapter, I will break down my approach for testing the Frontend, Backend, Machine Learning Module, the Database and the Storage layer, as well as evaluating the system at a higher level, ensuring that my application will remain set on the scope I designed in section 1.

5.2 Plan for Testing

Identify the approach for testing each part of the system
Provide test plan in structured manner

As I mentioned in the previous section, I designed the system with a modular approach in mind. As each element is isolated, each can be validated independently before I integrate it into the rest of the system.

The Frontend

- I will verify that the “Record” and “Stop Recording” buttons function as intended using MediaRecorder. Also ensuring a .wav file appears in storage.
- I will test the map initialises without issues, ensuring the map loads and Ireland is centred.
- I will test that the previous sample data appears in the list and populates map.
- I will test the popups which appear when the map markers are clicked is correct.

The Backend

- I will test that POST /api/predict/ stores the correct data as .wav in data storage.
- I will test that GET /api/predictions returns all previous predictions made as correctly formatted JSON.
- I will test that GET /api/regions/ returns all region polygons as correctly formatted GeoJSON.
- I will test that the predictions that the Machine Learning Module returns are serialised correctly in JSON.
- I will send invalid audio files through the API to ensure they are flagged.
- I will simulate incorrect file paths to test the systems response.
- I will test to ensure that the API returns valid HTTP status codes.

The Machine Learning Module

This module is the most experimental and error-prone section of the entire system, so therefore I have planned many small, repeatable tests.

- I will verify that the .wav files load correctly using Librosa.
- I will check all the audio samples are resamples to the same rate (16000Hz).
- I will ensure all audio is converted to mono to ensure uniformity.
- I will perform tests to ensure the extracted MFCC arrays have the correct **form**.
- I will ensure the selected classifiers return predictions without errors.
- I will test known coordinates of test data vs the predictions coordinate results to compare.
- I will attempt to classify corrupted/problematic audio samples. Other problems include Non-Irish English samples, silent samples.

The Database

- I will confirm Django connect to PostgreSQL and PostGIS inside of Docker.
- I will use PgAdmin to inspect the tables and created schemas.
- I will ensure that post-prediction that all data columns in **Prediction** table (columns id, audio_file_path, timestamp, predicted_coordinates and confidence_score) populate correctly.
- I will confirm this data matches the Frontend results.
- I will ensure the result of GET /api/regions/ returns data that matches the database table **Region**.
- I will ensure all data returns in the correct JSON and GeoJSON structure.

The Storage Layer

- I will ensure all audio files are stored properly within the Docker mounted storage.
- I will confirm the file paths in the database link correctly to the stored files.

If any of these tests fail, I will ensure the system fails gracefully and returns correct error messages.

5.3 Plan for Evaluation

As I developed my application with a modular architecture, to evaluate my system I must analyse each section independently, and after, the system in its entirety, all modules connected.

First I will evaluate each module:

The Machine Learning Module

Currently, my Machine Learning Module is a RandomForest model trained on the mini-Dáil dataset.

- Accuracy and Performance
I will test the RandomForest classifier by dividing the test into training and testing data, examining how accurate the model is on the unseen testing data.

- **Prediction Consistency**
I will run multiple predictions on the same input files to confirm that the classifier behaves consistently and does not return contradictory outputs across repeated runs.
- **MFCC Feature Validation**
I will ensure that all extracted MFCC arrays have the correct shape, no missing values, and consistent preprocessing (mono audio, 16000Hz sample rate).
- **Known-Origin Audio Testing**
I will evaluate the model using audio samples of known speaker origin, checking whether the predicted coordinates make sense for that region.
- **Classifier Reliability**
At this stage I will solely use the RandomForest classifier, ensuring that it is reliable with my current dataset.

The Frontend

My current frontend prototype loads a Leaflet map, displays the markers, and uploads audio.

- **Map Accuracy**
I will confirm that the predicted coordinates returned from the backend appear correctly on the map, and that popups show the correct metadata.
- **Markers**
I will check that all previously stored predictions load at startup and appear as markers in the correct locations.
- **Usability**
In the prototype, I want to see that the user can upload speech data and receive a result on the map.

The Backend

The Backend currently handles file uploads, JSON responses, and database writes using SQLite.

- **API Response Evaluation**
I will ensure that POST /api/predict/ and GET /api/predictions/ return correct JSON, the expected fields, and the appropriate HTTP status codes.
- **Error Handling**
I will test how the backend responds to any invalid or duplicate audio files.
- **Database Consistency**
I will ensure that all data within the database matches the data within the popups on the Frontend.
- **Storage Integrity**
I will make sure that the .wav files stored in storage have a corresponding row in the database with a matching file path.

The Whole System

Finally, I will evaluate the entire system, ensuring that the prototype work-flow works as intended:

Audio is uploaded > stored in backend > MFCC Features are extracted > ML makes prediction > Prediction stored in Database > The database is dynamically visualised on the Frontend.

I will ensure that the entire pipeline works as intended, and returns repeatable results when the same file is uploaded when the system is in a fresh-state.

5.4 Conclusions

In this chapter, I outlined my clear and structured approach for testing and evaluating my system. By separating each module of the system, I created a testing plan that allows each component to be verified independently before everything is integrated together. This modular testing will help identify issues early, reduce the complexity of debugging, and ensure that the system behaves consistently as I add new features and improvements.

I then detailed how I will evaluate the overall performance of the system, with my focus being on accuracy and reliability of the Machine Learning Model, the responsiveness of the Backend, the usability of the Frontend, and the spatial and relational data stored in the Database.

6. System Prototype

As least 2 pages, but as many as you like (with lots of code samples).

6.1 Introduction

For the interim stage of my project, I focused on creating a functional prototype that demonstrates the core behaviour of the full system. This project is designed using a modular architecture, so I implemented early versions of each component: the Frontend web-map, the Django Backend and REST API, the Machine Learning pipeline for accent classification, and the PostgreSQL/PostGIS Database for storing the predictions.

My goal starting this prototype was not to produce perfect predictions or a clean user interface, but instead I wanted to show that the main workflow of the system functions. The current design flow is: a user submits audio, the backend receives and stores it, the machine learning module processes and classifies it, the result is saved to the database, and the frontend displays the prediction on a map of Ireland.

This section presents the code samples, tools, and techniques I used to build this initial prototype, and shows how each module integrates with the others. It provides evidence that the system is technically feasible and that the architecture defined in earlier chapters is working as intended.

6.2 Prototype Development

Frontend

I designed the Frontend prototype with as basic a web page as was needed as this was not my focus at this stage. Upon loading the application the, a simple Leaflet.js map centred on Ireland is displayed on screen, with previous predictions shown on the map.

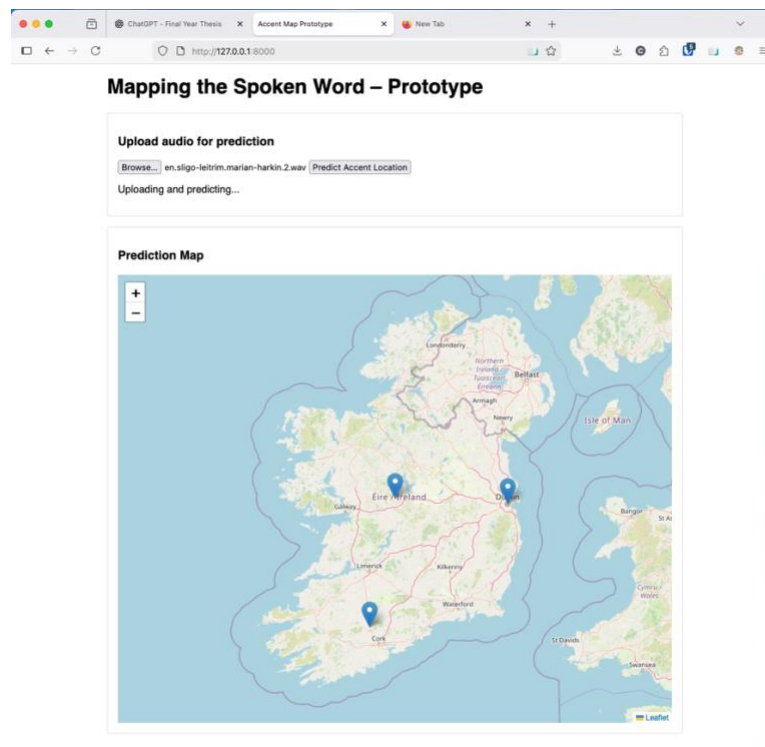


Figure 1 - Here is the current Frontend as it processes a .wav file for prediction. The markers shows previous prediction locations.

```
<script>
// Init map
var map = L.map('map').setView([53.4, -8.2], 7);
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
  maxZoom: 19,
}).addTo(map);

var markersLayer = L.layerGroup().addTo(map);

function loadPredictions() {
  fetch("/api/predictions/")
    .then(res => res.json())
    .then(data => {
      markersLayer.clearLayers();
      data.forEach(p => {
        L.marker([p.lat, p.lng])
          .addTo(markersLayer)
          .bindPopup("Prediction #" + p.id + "<br>" +
            "Lat: " + p.lat.toFixed(4) +
            "<br>Lng: " + p.lng.toFixed(4) +
            "<br>Confidence: " + (p.confidence ?? 0).toFixed(2));
      });
    })
    .catch(err => console.error("Error loading predictions:", err));
}

loadPredictions();
```

Figure 2 - Here the code initialising the map on the Frontend, as well as any markers of previous predictions that need to be displayed on the map.

```

// Handle form submission
document.getElementById("upload-form").addEventListener("submit", function(e) {
  e.preventDefault();
  var fileInput = document.getElementById("audio-input");
  var statusEl = document.getElementById("status");

  if (!fileInput.files.length) {
    statusEl.textContent = "Please select a .wav file first.";
    return;
  }

  var formData = new FormData();
  formData.append("audio", fileInput.files[0]);

  statusEl.textContent = "Uploading and predicting...";

  fetch("/api/predict/", {
    method: "POST",
    body: formData
  })
  .then(res => res.json())
  .then(data => {
    if (data.error) {
      statusEl.textContent = "Error: " + data.error;
    } else {
      statusEl.textContent = "Prediction complete. Label: " + data.label +
        " (" + data.lat.toFixed(4) + ", " +
        data.lng.toFixed(4) + ")";
      loadPredictions();
    }
  })
  .catch(err => {
    console.error(err);
    statusEl.textContent = "An error occurred.";
  });
});

```

Figure 3 - Here is the code for the data upload section of the Frontend.

Backend

I built my Backend using Django REST API as planned. I have just two API endpoints at this time:

- POST /api/predict – for uploading the audio for processing.
- GET /api/predictions/ – for retrieving stored predictions for displaying.

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', api_views.map_view, name='map'), # homepage = map
    path('api/predict/', api_views.predict_view, name='api_predict'),
    path('api/predictions/', api_views.predictions_view, name='api_predictions'),
]

```

Figure 4 - REST APIs from accentmap/urls


```

# API: upload audio → prediction
@csrf_exempt
def predict_view(request):
    if request.method != "POST":
        return JsonResponse({"error": "POST required"}, status=400)

    if 'audio' not in request.FILES:
        return JsonResponse({"error": "No audio file provided"}, status=400)

    audio_file = request.FILES['audio']
    save_dir = os.path.join(settings.MEDIA_ROOT, "audio_storage")
    os.makedirs(save_dir, exist_ok=True)
    save_path = os.path.join(save_dir, audio_file.name)

    # Save the file
    with open(save_path, "wb+") as dest:
        for chunk in audio_file.chunks():
            dest.write(chunk)

    # Run ML prediction
    (lat, lng), confidence, label = ml_module.predict(save_path)

    # Save in DB
    pred = Prediction.objects.create(
        audio_file_path=save_path,
        latitude=lat,
        longitude=lng,
        confidence_score=confidence,
    )

    return JsonResponse({
        "id": pred.id,
        "lat": lat,
        "lng": lng,
        "confidence": confidence,
        "label": label,
    })

```

Figure 5 - Code of the POST /api/predict/ endpoint.

This endpoint stores the audio, calls the Machine Learning Module, saves the prediction, and returns it to the Frontend.

```

# API: list predictions for map
def predictions_view(request):
    preds = Prediction.objects.order_by("-timestamp")[:100]
    data = [
        {
            "id": p.id,
            "lat": p.latitude,
            "lng": p.longitude,
            "confidence": p.confidence_score,
            "timestamp": p.timestamp.isoformat()
        }
        for p in preds
    ]
    return JsonResponse(data, safe=False)

```

Figure 6 - Code of the GET /api/predictions/ endpoint.

Machine Learning Module

For the interim version, I implemented a basic pipeline using Librosa to extract MFCCs and a RandomForest classifier to produce the prediction.

```

import librosa
import numpy as np
import joblib
import os

MODEL_PATH = os.path.join(os.path.dirname(__file__), "rf_model.pkl")
model = joblib.load(MODEL_PATH)

REGION_TO_COORDS = {
    "Leinster": (53.3498, -6.2603),
    "Munster": (52.0667, -8.6333),
    "Connacht": (53.2707, -9.0568),
    "Ulster": (54.5970, -5.9300),
}

def extract_mfcc(path):
    y, sr = librosa.load(path, sr=16000, mono=True)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    return np.mean(mfcc.T, axis=0)

def predict(path):
    features = extract_mfcc(path).reshape(1, -1)
    label = model.predict(features)[0] # e.g. "Leinster"
    coords = REGION_TO_COORDS.get(label, (53.4, -8.2))
    confidence = 0.75
    return coords, confidence, label

```

Figure 7 - This shows the code of the entire `ml_module.py`. The data is extracted by Librosa, and the Predictions are done using the trained data within `rf_model.pkl`.

This classifier is trained on the smaller Dáil dataset I used for this prototype.

		precision	recall	f1-score	support
3	Connaught	1.00	0.60	0.75	5
4	Leinster	0.74	0.91	0.82	22
5	Munster	0.78	0.64	0.70	11
6	Ulster	0.00	0.00	0.00	1
8	accuracy			0.77	39
9	macro avg	0.63	0.54	0.57	39
10	weighted avg	0.77	0.77	0.75	39
12	Per-class rows:				
13	Support = how many test items of that class there were (e.g., 22 Leinster clips in test).				
14	Recall ("sensitivity") = out of the true items of this class, what fraction did we correctly catch?				
15	recall = TP / (TP + FN)				
16	Leinster 0.91 means 91% of true Leinster clips were predicted as Leinster.				
17	Precision = of the items we predicted as this class, what fraction were actually correct?				
18	precision = TP / (TP + FP)				
19	Connaught 1.00 means every time the model predicted Connaught, it was right – but note support is small.				
20	F1 = harmonic mean of precision and recall. Balances both.				
21	F1 = 2 * (precision * recall) / (precision + recall)				
23	Global rows:				
24	accuracy 0.77 = overall % correct across all classes.				
25	macro avg = unweighted mean across classes (each class counts equally). Sensitive to small classes (Ulster drags it down).				
26	weighted avg = mean weighted by support (big classes like Leinster dominate). Often higher when the dataset is imbalanced.				

Figure 8 - This image shows the results of a the RandomForest Classifier, getting a 77% accuracy, which is fantastic for such a small sample set.

Database Prototype

For this interim report, I used Django's built in SQLite database, which allowed me to store the prediction results, audio file-paths and the coordinates without having to configure a full PostgreSQL/PostGIS system at this stage. SQLite is very lightweight, perfect for the early demo in this stage.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Figure 9 - The code for the Database currently used. Found within accentmap/settings.py

Storage Layer

The Storage Layer is currently stored locally, within accentmap/media/audio_storage. This would not work for the full application, but it is sufficient for the current Prototype.

6.3 Results

At this interim stage, I carried out a series of prototype tests based on the testing plan outlined in Chapter 5. As I designed the system with a modular architecture, I was able to test each component independently.

Below is a breakdown of the tests that were completed, as well as the tests that will be finalised during the next phase of development.

Frontend Tests

Completed Tests:

- I will ensure a .wav file appears in storage.

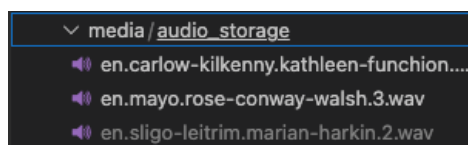


Figure 10 - Image of .wav files saved in storage uploaded by the user.

- I will test the map initialises without issues, ensuring the map loads and Ireland is centred.

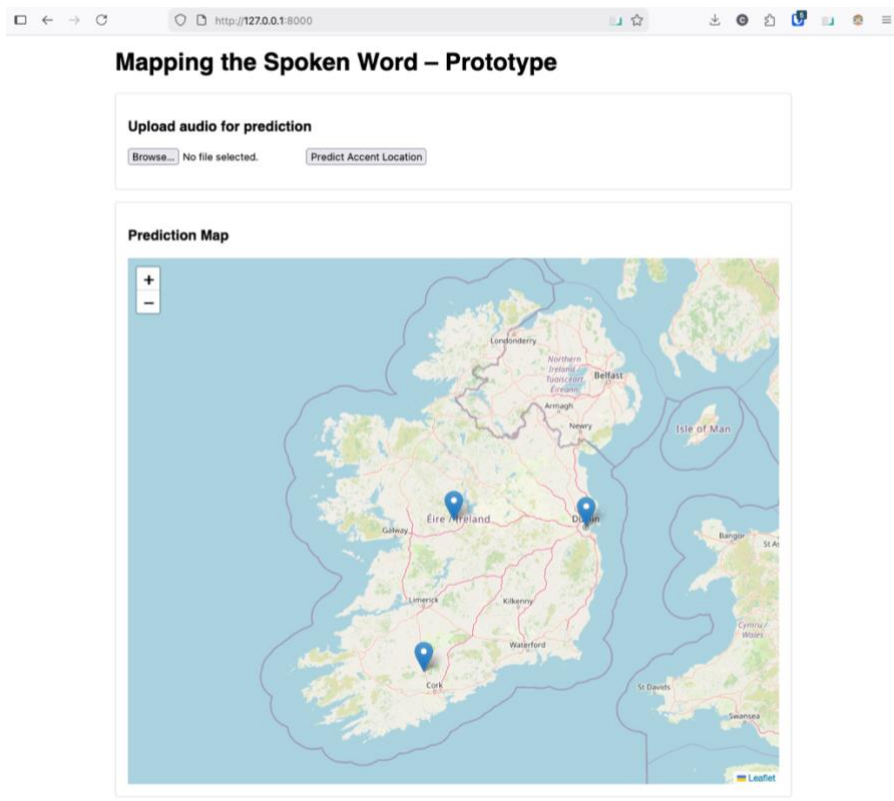


Figure 11 - An image showing the Frontend, with Ireland centred upon loading.

- I will test that the previous sample data populates map.

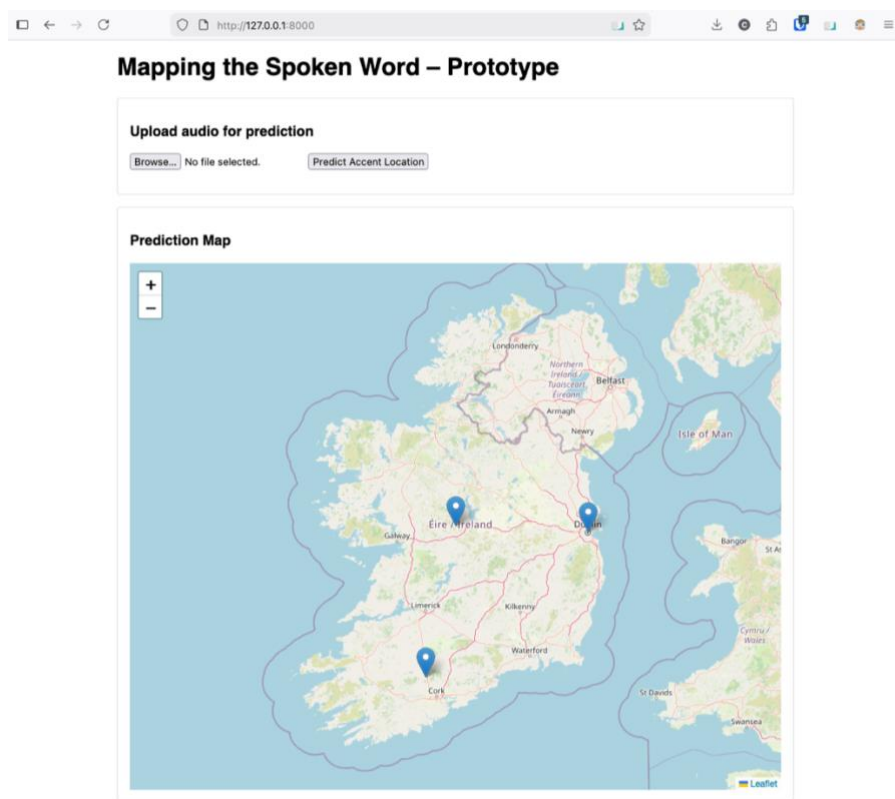


Figure 12 - Image showing previous predictions appearing as markers on the map.

- I will test the popups which appear when the map markers are clicked is correct.

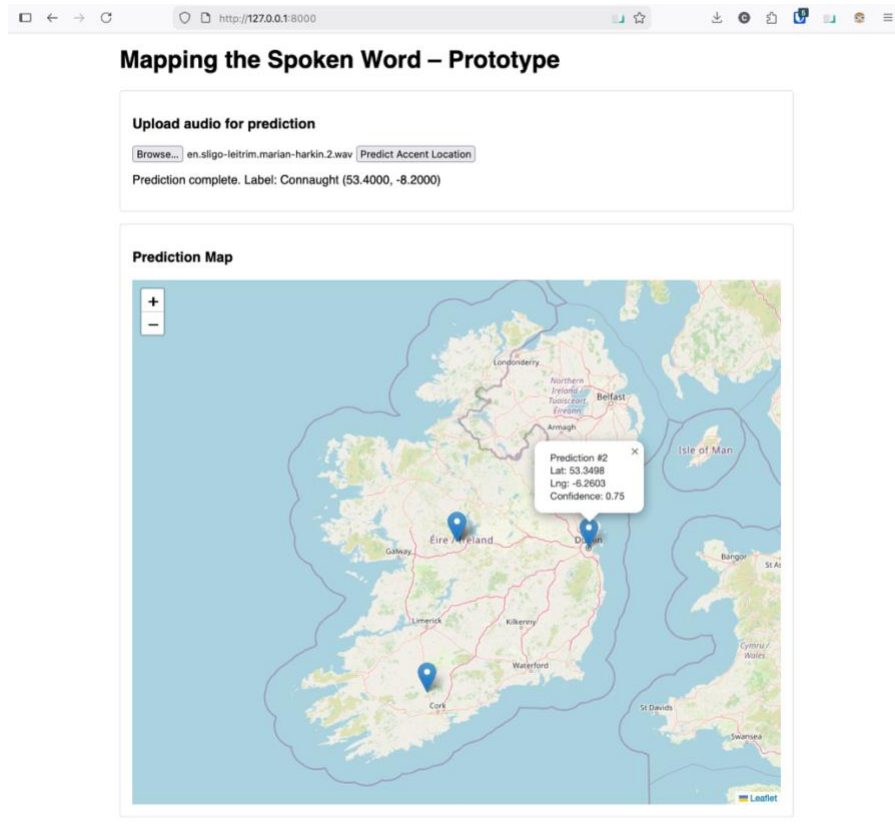


Figure 13 - Image showing the popup appear as a marker is clicked.

Incomplete Tests:

- I will verify that the “Record” and “Stop Recording” buttons function as intended using MediaRecorder.
- I will test that the previous sample data appears in the list.

Backend Tests

Completed Tests:

- I will test that POST /api/predict/ stores the correct data as .wav in data storage.

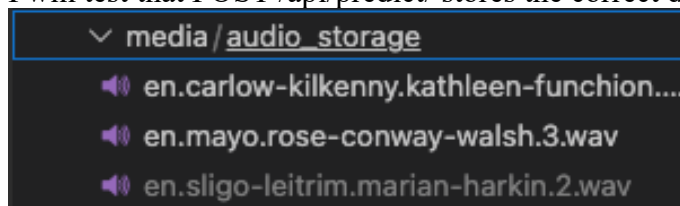


Figure 14 - Data stored as .wav files.

- I will test that GET /api/predictions/ returns all previous predictions made as correctly formatted JSON.

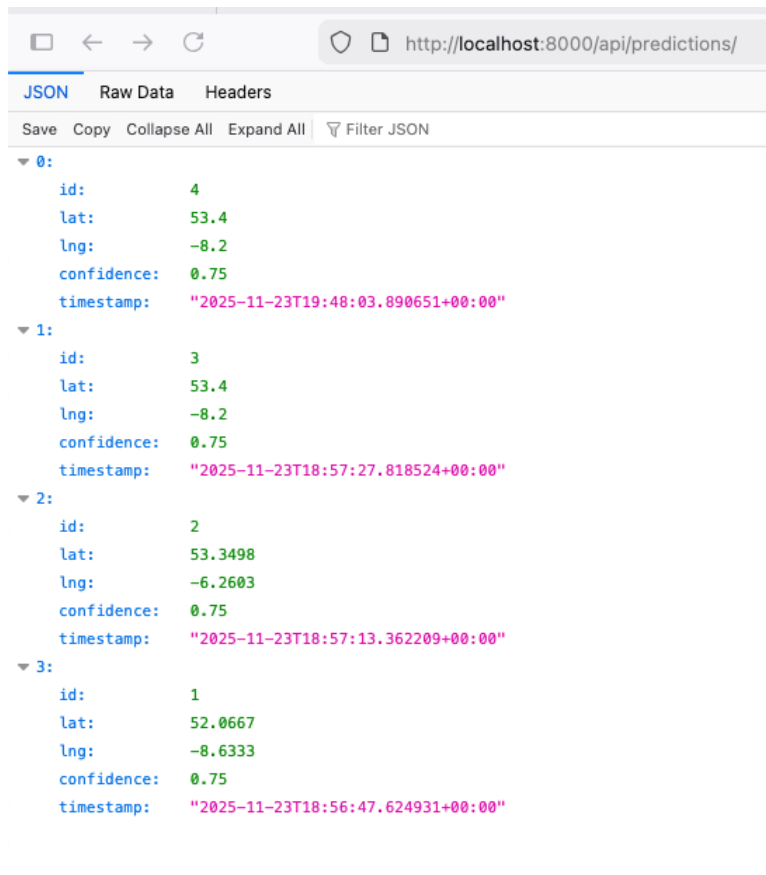


Figure 15 - Image showing results of `/api/predictions/` as correctly formatted JSON.

- I will test that the predictions that the Machine Learning Module returns are serialised correctly in JSON.

```
# API: list predictions for map
def predictions_view(request):
    preds = Prediction.objects.order_by("-timestamp")[:100]
    data = [
        {
            "id": p.id,
            "lat": p.latitude,
            "lng": p.longitude,
            "confidence": p.confidence_score,
            "timestamp": p.timestamp.isoformat()
        }
        for p in preds
    ]
    return JsonResponse(data, safe=False)
```

Figure 16 - Image showing the Django View serialising the response as valid JSON.

- I will send invalid audio files through the API to ensure they are flagged.

Mapping the Spoken Word – Prototype

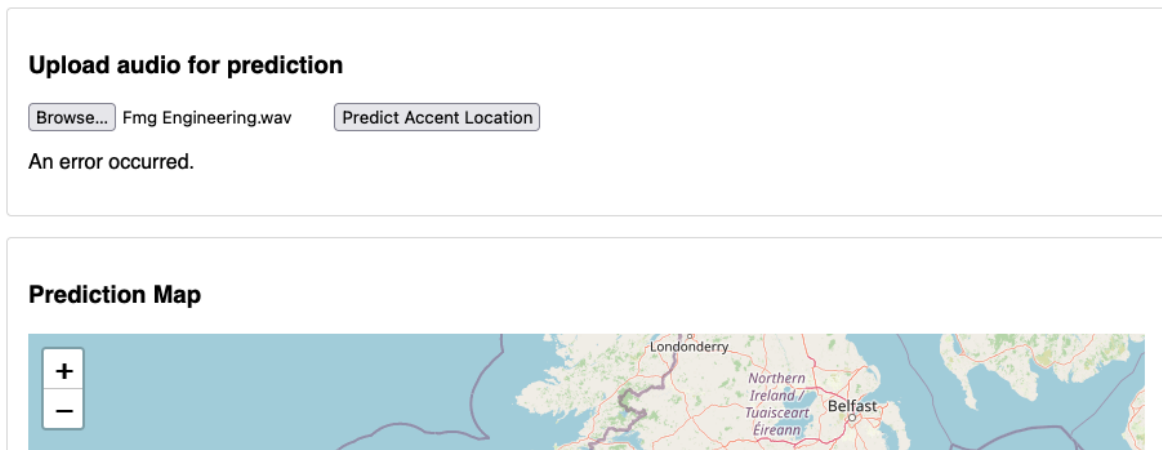


Figure 17 - Frontend showing an error after a file with the same name as a already saved .wav file in storage.

- I will test to ensure that the API returns valid HTTP status codes.

```
[23/Nov/2025 19:46:28] "GET /api/predictions/ HTTP/1.1" 200 330
[23/Nov/2025 19:48:03] "POST /api/predict/ HTTP/1.1" 200 77
[23/Nov/2025 19:48:03] "GET /api/predictions/ HTTP/1.1" 200 436
[23/Nov/2025 20:51:39] "GET /api/predictions/ HTTP/1.1" 200 436
Not Found: /favicon.ico
[23/Nov/2025 20:51:39] "GET /favicon.ico HTTP/1.1" 404 2893
```

Figure 18 - Terminal showing HTTP status codes while app running.

Incomplete Tests:

- I will test that GET /api/regions/ returns all region polygons as correctly formatted GeoJSON.
- I will send invalid audio files through the API to ensure they are flagged.

Machine Learning Module Tests

Completed Tests:

- I will verify that the .wav files load correctly using Librosa.
- I will check all the audio samples are resamples to the same rate (16000Hz).
- I will ensure all audio is converted to mono to ensure uniformity.

```
Prototype1 > PrototypeScripts > verify_audio.py > ...
1  import librosa
2  import pandas as pd
3  import os
4
5  meta = pd.read_csv("/Users/cianan/Documents/GitHub/FYP/Prototype1/data/metadata.csv")
6  audio_folder = "/Users/cianan/Documents/GitHub/FYP/Prototype1/data/audio/"
7
8  for i, row in meta.iterrows():
9      path = os.path.join(audio_folder, row['filename'])
10     y, sr = librosa.load(path, sr=None, mono=True)
11     print(f"{row['filename']}: {sr} Hz, length {len(y)/sr:.2f}s")
12
```

Figure 19 - Image showing verify_audio.py.

```

6 en.cavan-monaghan.heather-humphreys.1.wav: 16000 Hz, length 3.68s
7 en.cavan-monaghan.heather-humphreys.2.wav: 16000 Hz, length 5.49s
8 en.cavan-monaghan.heather-humphreys.3.wav: 16000 Hz, length 4.55s
9 en.cavan-monaghan.heather-humphreys.4.wav: 16000 Hz, length 7.26s
10 en.cavan-monaghan.heather-humphreys.5.wav: 16000 Hz, length 5.04s
11 en.clare.violet-anne-wynne.1.wav: 16000 Hz, length 1.88s
12 en.clare.violet-anne-wynne.2.wav: 16000 Hz, length 3.81s
13 en.clare.violet-anne-wynne.3.wav: 16000 Hz, length 3.11s
14 en.clare.violet-anne-wynne.4.wav: 16000 Hz, length 3.84s
15 en.clare.violet-anne-wynne.5.wav: 16000 Hz, length 3.53s
16 en.cork-east.pat-buckley.1.wav: 16000 Hz, length 6.48s
17 en.cork-east.pat-buckley.2.wav: 16000 Hz, length 2.35s
18 en.cork-east.pat-buckley.3.wav: 16000 Hz, length 6.01s
19 en.cork-east.pat-buckley.4.wav: 16000 Hz, length 5.62s
20 en.cork-east.pat-buckley.5.wav: 16000 Hz, length 4.41s
21 en.cork-north-central.mick-barry.1.wav: 16000 Hz, length 5.41s
22 en.cork-north-central.mick-barry.2.wav: 16000 Hz, length 2.80s
23 en.cork-north-central.mick-barry.3.wav: 16000 Hz, length 3.55s

```

Figure 20 - Image from `verify_audio_results.txt`, showing all .wav files loaded correctly and as 16000Hz and in mono format.

- I will perform tests to ensure the extracted MFCC arrays have the correct form.

```

Prototype1 > PrototypeScripts > test_mfcc_shape.py > ...
1 import librosa
2 import numpy as np
3
4 def test_mfcc_shape(path):
5     y, sr = librosa.load(path, sr=16000, mono=True)
6     mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
7
8     print("MFCC shape:", mfcc.shape)
9     print("Data type:", type(mfcc))
10    print("Contains NaN:", np.isnan(mfcc).any())
11    print("Contains Inf:", np.isinf(mfcc).any())
12
13    # Expected: 13 coefficient rows
14    if mfcc.shape[0] != 13:
15        raise ValueError("MFCC extraction error: Expected 13 coefficients.")
16
17    # Expected: at least one frame
18    if mfcc.shape[1] < 1:
19        raise ValueError("MFCC extraction error: No frames extracted.")
20
21    return "MFCC test passed!"
22
23 print(test_mfcc_shape("/Users/cianan/Documents/GitHub/FYP/Prototype1/data/audio/en.carlow-kilkenny.kathleen-funchion.1.wav"))
24

```

Figure 21 - Image showing `test_mfcc_shape.py`.

```

(fyp_env_py313) cianan@Cianans-MacBook-Pro FYP % ./ts/test_mfcc_shape.py
MFCC shape: (13, 184)
Data type: <class 'numpy.ndarray'>
Contains NaN: False
Contains Inf: False
MFCC test passed!

```

Figure 22 - Image showing successful result of `test_mfcc_shape.py`.

- I will ensure the selected classifiers return predictions without errors.

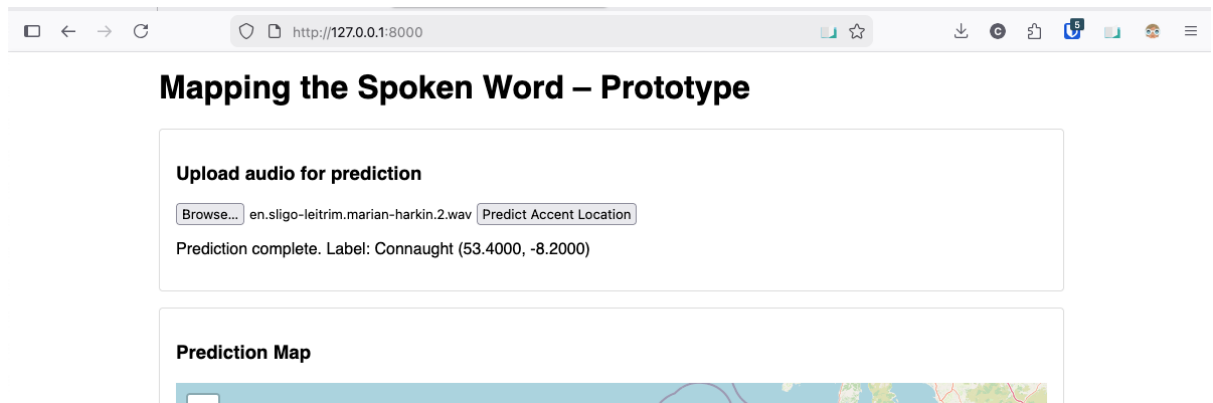


Figure 23 - Image showing prediction made via file upload with no errors.

- I will test known coordinates of test data vs the predictions coordinate results to compare.

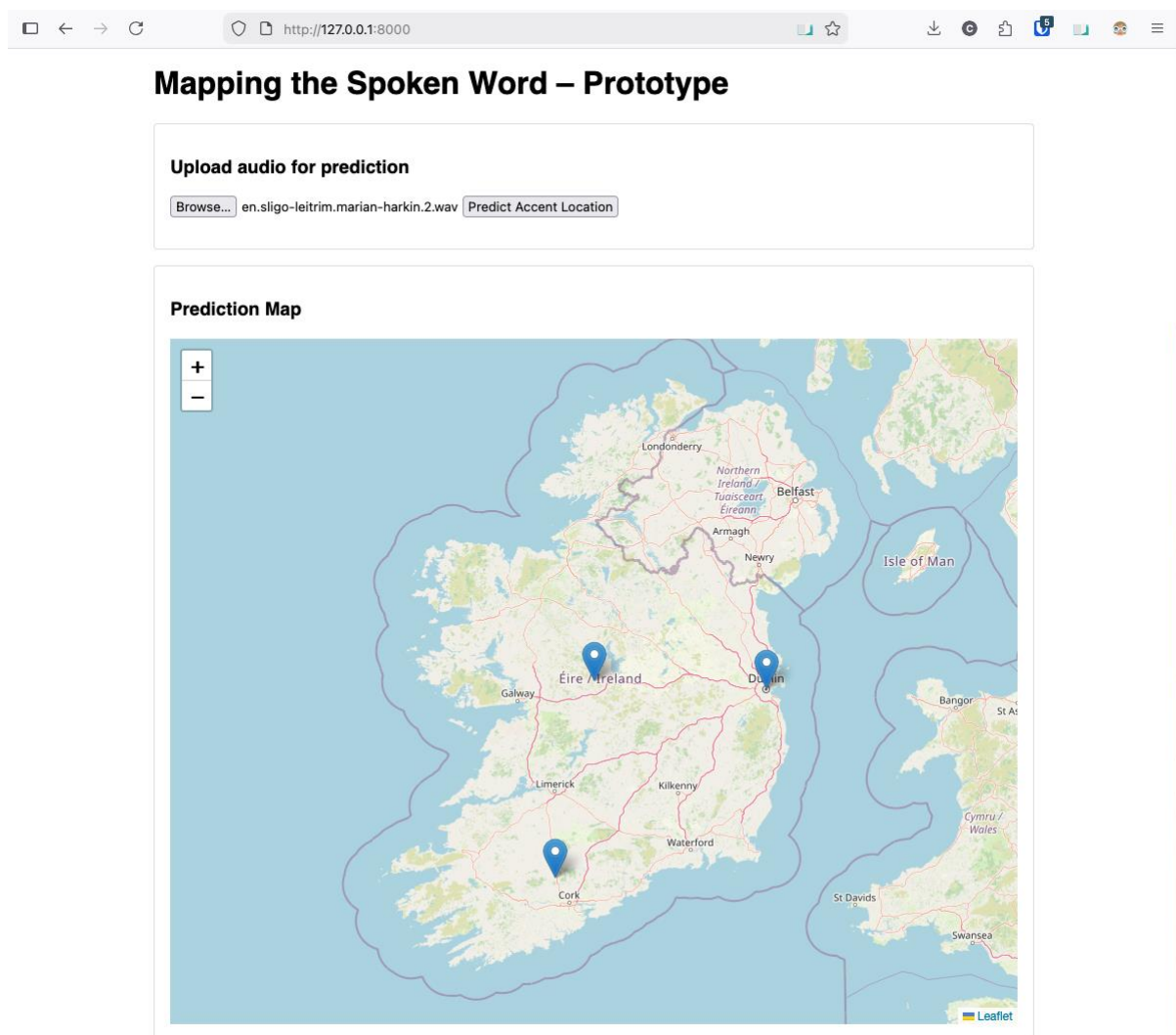


Figure 24 - Image showing upload of file of known speaker origin.

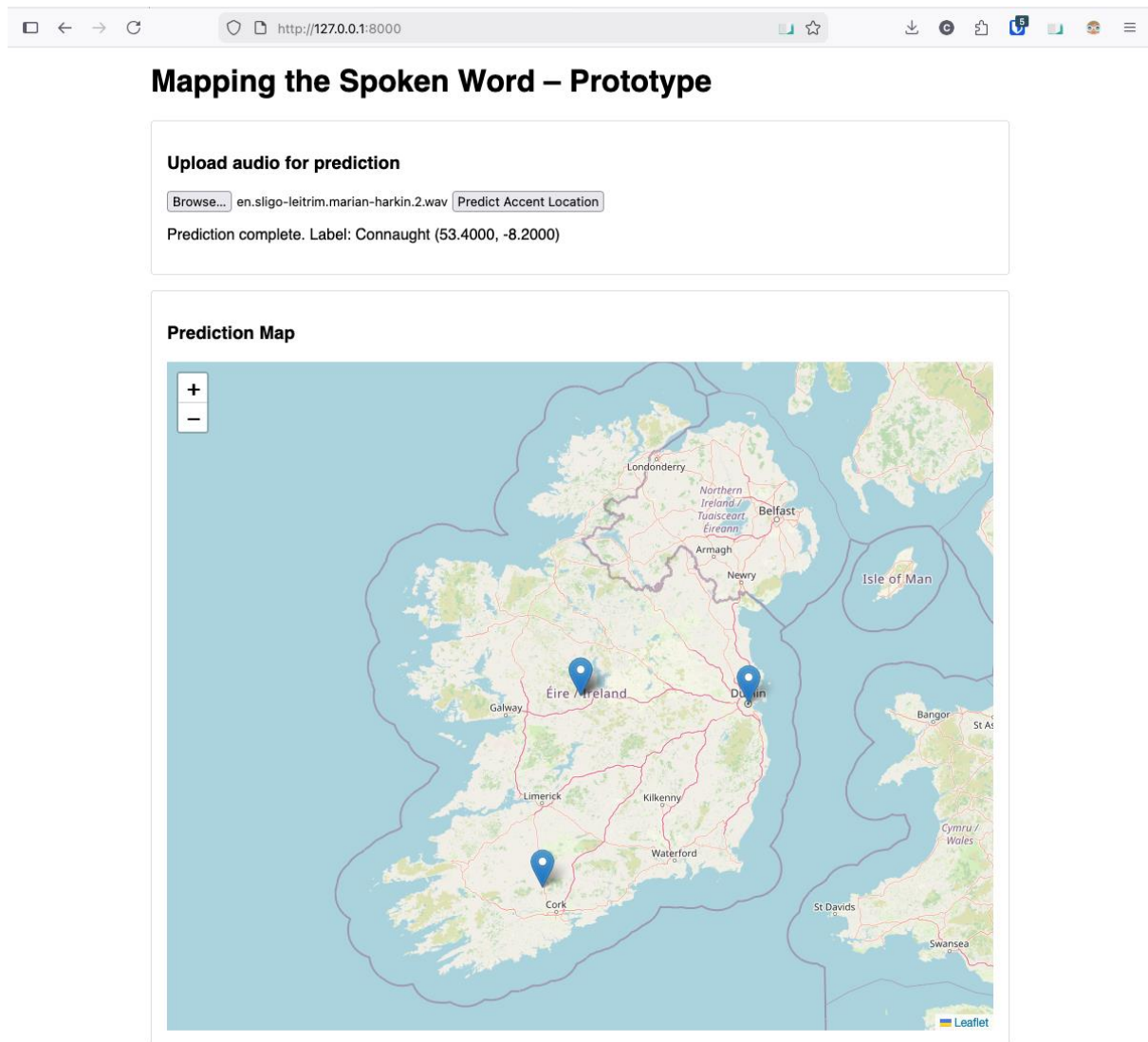


Figure 25 - Image showing correct result of file with known speaker origin.

Incomplete Tests:

- I will attempt to classify corrupted/problematic audio samples. Other problems include Non-Irish English samples, silent samples.

Database/Storage Layer Tests

Completed Tests:

- I will confirm this data matches the Frontend results.

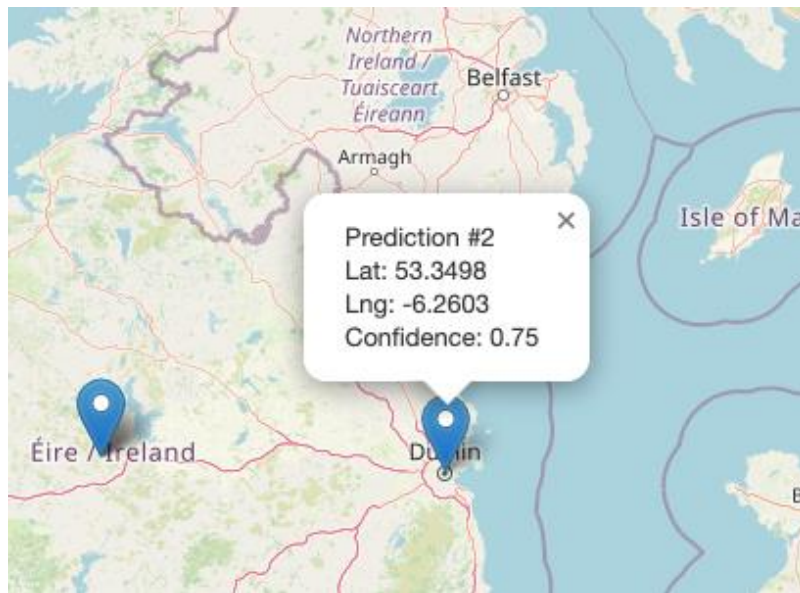


Figure 26 - Image of Frontend popup showing same data as database JSON below.

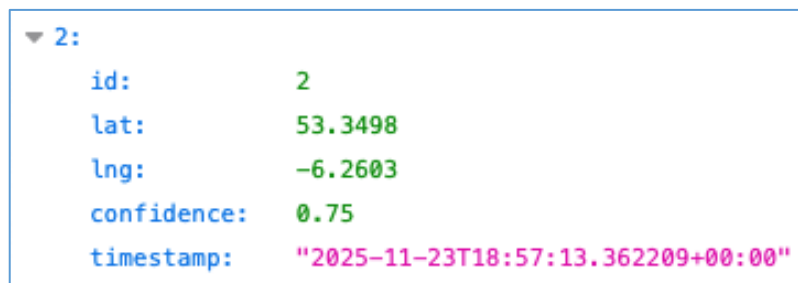


Figure 27 - Image of Database JSON showing same data as the Frontend popup above.

Incomplete Tests:

- I will confirm Django connect to PostgreSQL and PostGIS inside of Docker.
- I will use PgAdmin to inspect the tables and created schemas.
- I will ensure that post-prediction that all data columns in **Prediction** table (columns id, audio_file_path, timestamp, predicted_coordinates and confidence_score) populate correctly.
- I will ensure the result of GET /api/regions/ returns data that matches the database table **Region**.
- I will ensure all data returns in the correct JSON and GeoJSON structure.

6.4 Evaluation

Frontend

The Frontend passed all basic tests. The Leaflet map always initialised correctly centred on Ireland, all previous predictions appeared as markers on the map in the correct locations. All the popups had accurate metadata. The file upload worked well, with dynamically updating process information and a predicted tidy result. The missing features will be developed later but I am happy how it is at this point in time.

Backend and Database

The Backend APIs I implemented behaved as expected, with accurate and well-structured JSON being returned, as well as accurate HTTP status codes within the terminal. The error handling was proved to work for invalid file uploads. The data within the database was accurate and matched what was displayed to the user, confirming the pipeline works consistently.

Machine Learning Module

My MFCC extraction tests showed all data was correctly converted to Mono, 16000Hz and with the correct MFCC array shape. The RandomForest classifier worked well, achieving a 77% accuracy rate with such a small dataset. I was able to show that the model produced the same results for the same data consistently.

6.5 Conclusions

In this chapter I showed that the system is already up and running, in a basic but informative way. I have a working pipeline, where a user can upload data, the backend will process it, the machine learning model will analyse it and return a prediction, the result is stored in the database and the frontend dynamically updates. Each part has been tested separately, and the system's pipeline is functional and works reliably.

My system is in very early stages, but the prototype proves that the planning, designing and research that I completed in this report came together to create an app with architecture that works. I think the system is in a good position to move into the future development stage.

7. Issues and Future Work

7.1 Introduction

Purpose of the section

7.2 Issues and Risks

Dataset Limitations

By far the largest issue my project has faced while developing the limited amount of Hiberno-English data with reliable metadata or referenceable information. Although I created my own regionally labled dataset by collecting and trimming ~1500 clips from the Dáil, the following issues remain:

- **Regional Imbalance**
Some regions have more clips than others, risking learning a geographic bias.
- **Demographic Skew**
The Dáil is supposed to be representative of the Irish peoples' interests, but the language spoken within politics tends to be more formal than what the average Hiberno-English speaker sounds like on a daily basis.
The number of men dominates the amount of women, with only 25% of TDs being women.
The average age of the 31st Dáil was 48.5, skewing the data towards older speakers.
- **Accent similarity**
Accents do not stop at geographic borders, meaning dialects may only differ very subtly across neighbouring regions, making data overlap and classification more difficult.

Overfitting Risk

Due to having to deal with such a small dataset, there is a significant risk of the model learning patterns that aren't related to the speakers accent, such as:

- **Microphone Quality**
Since the same microphones are consistently used, any imperfections may be learned by the machine.
- **Background noise in the Dáil**
The Dáil can be a very noisy place which remains within the recordings.
- **Phrasing used in Political Speeches**
The vocabulary used by the politicians can be repetitive and cause the model to learn false patterns.

Frontend Features

- **MediaRecorder**
The MediaRecorder has yet to be implemented, with users only being able to upload data as files.
- **Results List**
The results list has yet to be implemented.
- **Clunky User Interface**
The UI is clunky and unintuitive, making the application unattractive to users.

Time

A major risk in my project is time. There are many time-consuming elements, such as Data Collection, Metadata Creation, Machine Learning Model Testing, Application Coding, Frontend Refining and Report Writing of course.

7.3 Plans and Future Work

The Frontend

- Add MediaRecorded to allow for speech recording.
- Implement List Section.
- Improve layout with new UI.
- Allow users to relisten to audio.

Potential Future Frontend UI

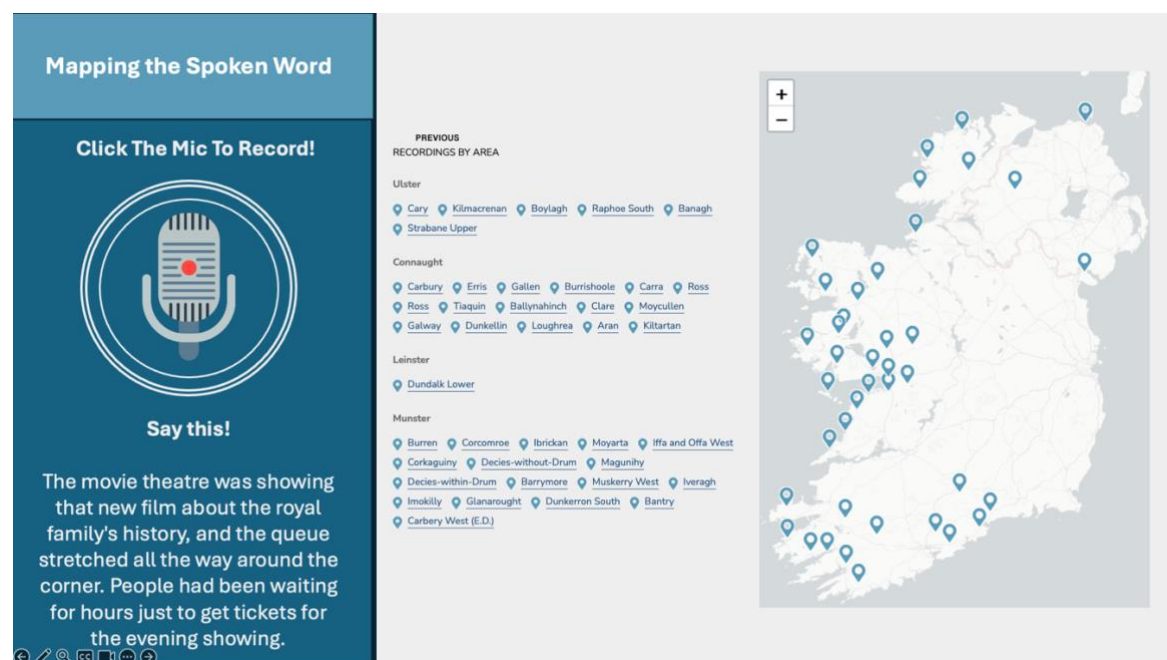


Figure 28 - Future UI Initial Screen.

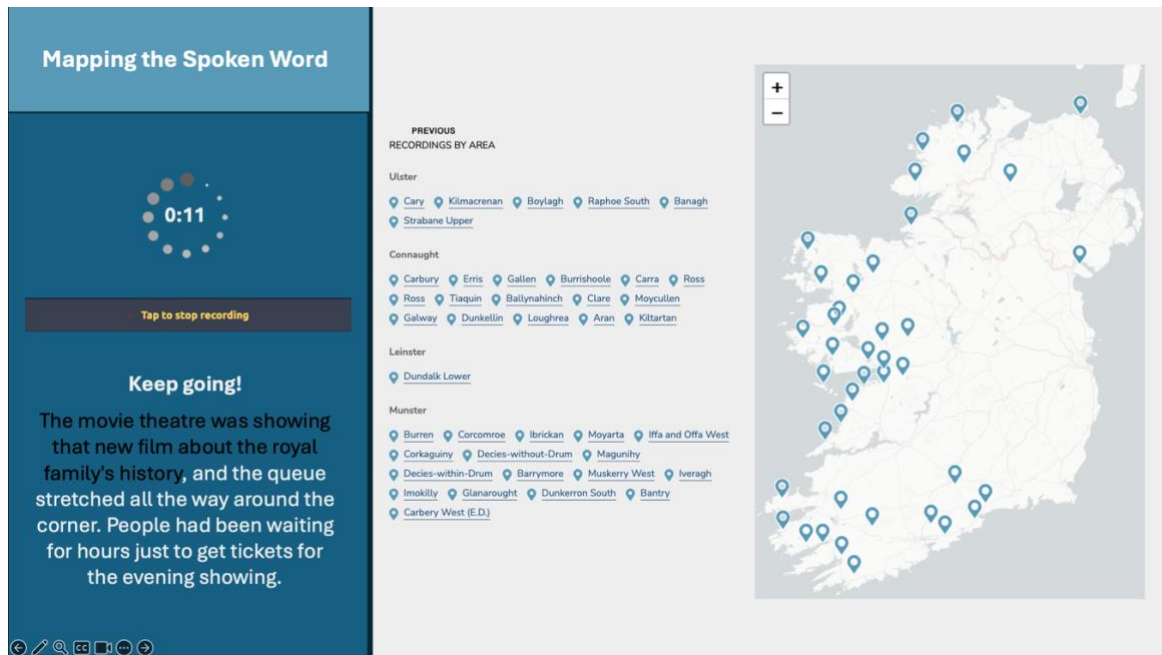


Figure 29 - Future UI Recording Sample Screen

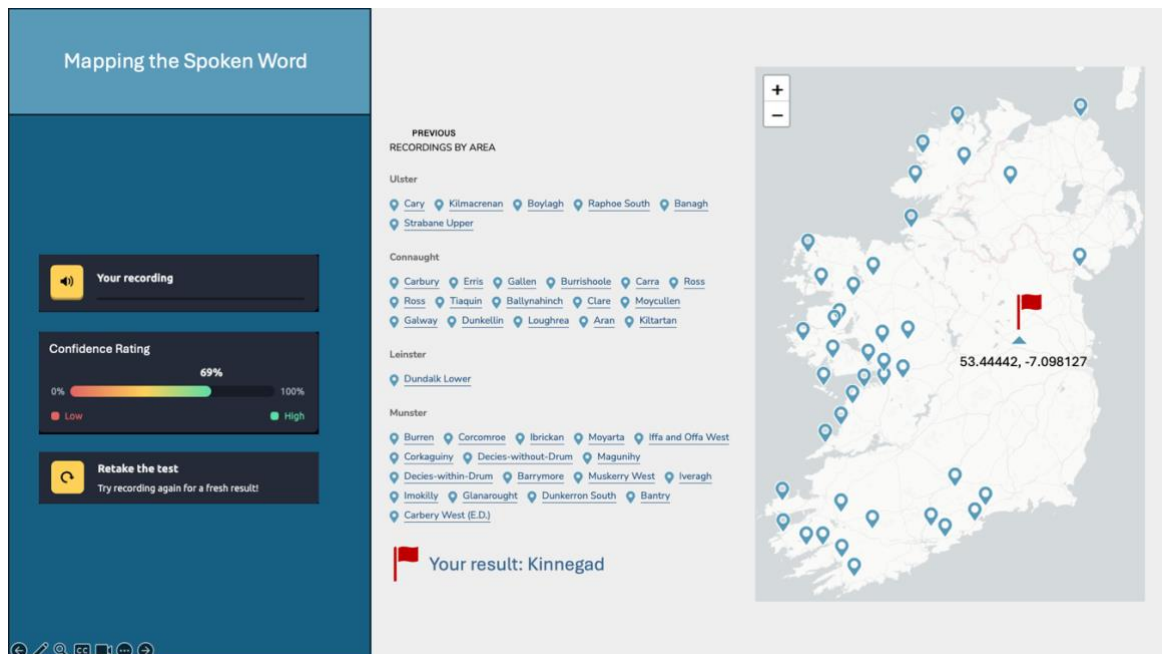


Figure 30 - Future UI Prediction Results Screen

The Backend

- Migrate from SQLite to PostgreSQL with a PostGIS extension.
- Containerise everything cleanly using Docker Compose.
- Add GET /api/regions/ with real geographic region boundaries.
- Improve the request validation and error handling.

The Machine Learning Module

- Expand the current dataset.
- Introduce more balanced samples across constituencies.
- Filter noise and research audio augmentation.

- Test more models (Wav2Vec2 (Baevski et al., 2020), kNN baseline).
- Build a confusion matrix to identify any weaker regions.
- Implement a word guide to guide the speaker on words more commonly used by politicians (as seen in Figure 29 above).
- Implement confidence rating system, as seen in Figure 30 above.
- Find solutions to speed up analysis.

The Database

- Import county/constituency polygons.
- Add indexes for faster spatial queries.
- Make region-level analytics for the final report.

7.3.1 Project Plan with GANTT Chart

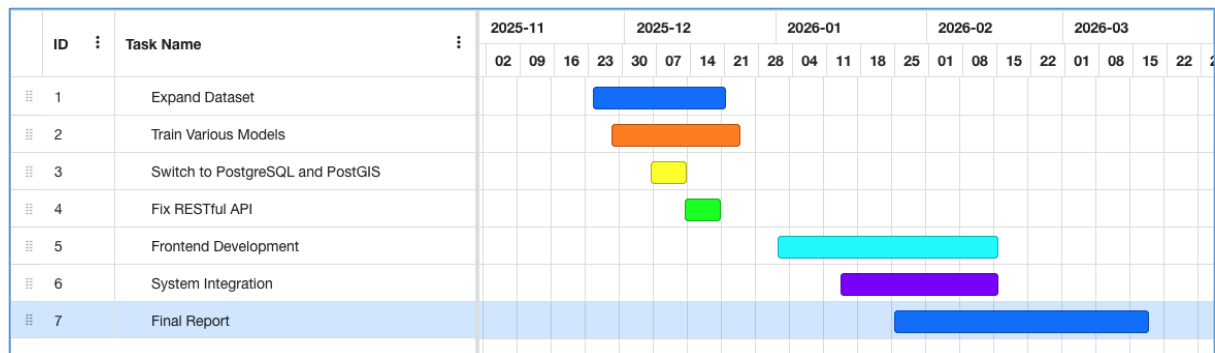


Figure 31 - Gantt Chart of Project Plan

References

- Baevski, A., Zhou, H., Mohamed, A. and Auli, M. (2020)** ‘wav2vec 2.0: A framework for self-supervised learning of speech representations’. *Advances in Neural Information Processing Systems*. Available at: <https://doi.org/10.48550/arXiv.2006.11477>
- Ding, D. (2016)** *Locallingual*. Available at: <https://www.locallingual.com>
- Django Software Foundation (2023)** *Django Documentation*. Available at: <https://docs.djangoproject.com>
- Esri (2023)** *ArcGIS Online: Web Mapping Platform*. Available at: <https://www.arcgis.com>
- Google Cloud (2023)** *Speech-to-Text Documentation*. Available at: <https://cloud.google.com/speech-to-text>
- Hickey, R. (no date)** *Irish English Studies: An historical overview*. Available at: [http://www.raymondhickey.com/Irish_English_Studies_\(Hickey\).pdf](http://www.raymondhickey.com/Irish_English_Studies_(Hickey).pdf)
- Journal of Engineering, IOSR (2014)** ‘An approach to extract feature using MFCC’. DOI: 10.9790/3021-04812125
- Leaflet (2023)** *Leaflet.js Documentation*. Available at: <https://leafletjs.com>
- McFee, B. et al. (2015)** ‘librosa: Audio and Music Signal Analysis in Python’. *Proceedings of the 14th Python in Science Conference*, pp. 18–25.
- Merkel, D. (2014)** ‘Docker: lightweight Linux containers’. *Linux Journal*, 239.
- OpenStreetMap Contributors (2023)** *OpenStreetMap*. Available at: <https://www.openstreetmap.org>
- PostGIS Project (2023)** *PostGIS 3.4 Manual*. Available at: <https://postgis.net/docs>
- RTÉ Archives (2025)** *Canúint*. Available at: <https://www.rte.ie/archives/category/archives/2025/0303/1499960-canuint/>
- Sabuj, H.H., Ahmed, A., Paul, N., Lamia, F. and Khan, F. (2023)** ‘A comparative study of machine learning classifiers for speaker’s accent recognition’. *2023 IEEE World AI IoT Congress (AIIoT)*, pp. 627–632. DOI: 10.1109/AIIoT58121.2023.10174511
- Taspinar, Y.S., Koklu, M. and Altin, M. (2020)** ‘Identification of the English accent spoken in different countries by the k-nearest neighbor method’. *International Journal of Intelligent Systems and Applications in Engineering*, 8(4), pp. 191–194.
- Vocal Image (2023)** *Vocal Image: Voice and Accent Training*. Available at: <https://vocalimage.ai>

von Münchhausen, L. (1591) *A German Visitor to Monaincha in 1591*. Celtic Digital Initiative, University College Cork. Available at: <https://celt.ucc.ie/published/T100083/>

- Appendix A: System Model and Analysis

Details of Requirements gathering and analysis method

- Appendix B: Design

Details of design approach used

System elements, components, classes

- Appendix C: Prompts Used with ChatGPT

- Appendix D: Additional Code Samples

Provide additional code samples
Organised by Logical Architecture

- Appendix E:

Other Appendix